This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

MTDiff: A Multi-Task Diffusion Accelerator With Bandwidth-Aware Memory Partition and Efficient Digital Compute-in-Memory

Jiaqi Zhou, Yiqi Jing, *Graduate Student Member, IEEE*, Yiyang Sun, Siyuan He, *Graduate Student Member, IEEE*, Le Ye, *Senior Member, IEEE*, and Tianyu Jia, *Member, IEEE*

***Abstract*—Initially applied to image synthesis, diffusion models have rapidly expanded into various content generation tasks, such as 3-D scene reconstruction and video generation, due to the superior generative quality. Based on our observations, the imbalanced operational intensities across diffusion model layers lead to suboptimal hardware utilization due to inadequate bandwidth optimization. Additionally, emerging 3-D and video generation tasks require computationally intensive multi-view consistency operations to ensure smooth transitions between perspectives and frames. In this work, we present *MTDiff*, a multi-task diffusion accelerator featuring three key innovations: 1) a dynamic bandwidth-aware memory partitioning scheme optimizing hardware utilization and reducing external memory accesses; 2) an efficient bitline (BL)-segmented compute-in-memory (CIM) macro together with a reuse-aware weight update scheduler; and 3) a hierarchical multi-frame consistency management (MFCM) skips redundant operations at frame and pixel levels to improve computational efficiency. Implemented using a 22-nm process, the *MTDiff* accelerator achieves a 49.74–60.81 TFLOPS/W system efficiency across various generative tasks. It delivers a 1.4× performance improvement and a 1.52× system energy efficiency enhancement compared to prior state-of-the-art (SOTA) diffusion accelerators.**

***Index Terms*—Diffusion model, digital compute-in-memory (CIM), memory partition, multi-task generation, visual consistency management.**

I. Introduction

With the advancement of artificial intelligence (AI), AI-generated content (AIGC) [1] has become a prominent research focus from various application domains [2], [3]. Diffusion [4], [5] is the mainstream model architecture for landmark image generation, for example, OpenAI's DALL-E 2 [6], DALL-E 3 [7], and stability AI's stable diffusion [8]. Beyond traditional image generation, diffusion models have been extended to diverse visual content generation tasks such as 3-D modeling [9], [10] and video generation [11], [12]. The advent of diffusion has also significantly propelled the development of large visual models (LVMs), expanding generative model outputs from language to the visual domain.

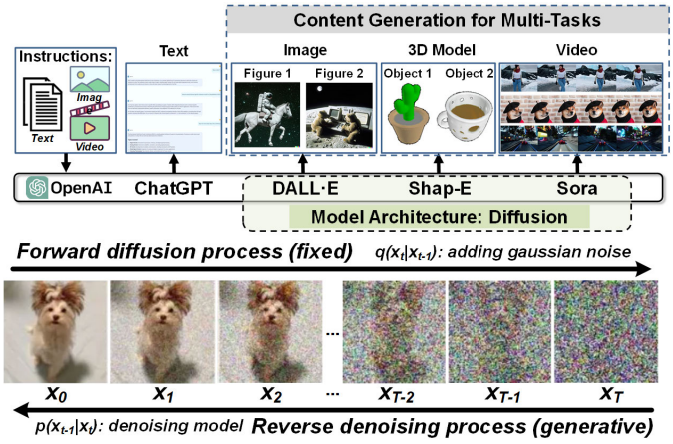


Fig. 1. Overview of multi-task content generation and diffusion model.

As shown in Fig. 1, diffusion models are probabilistic generative models with state-of-the-art (SOTA) generation fidelity and quality compared to traditional VAE [13] and GAN [14] models. The mechanism of diffusion principally involves two interconnected processes: the predefined forward diffusion process employs a sequence of forward transition kernels to progressively transform the original data distribution into a Gaussian distribution; the reciprocal denoising process utilizes trained U-Net [15] or diffusion transformer (DiT) [16] networks to gradually reverse the forward process via ordinary differential equation (ODE) or stochastic differential equation (SDE) [17] to reconstruct data. To generate high-quality visual content, such as images or videos, the diffusion models require multiple iterative denoising steps to construct the generation pipeline.

Despite demonstrating exceptional capabilities, diffusion models face challenges in hardware deployment. In particular, we observe that their inference exhibits the following characteristics: 1) wide range bandwidth demands across model layers due to heterogeneous feature map resolutions and channel dimensions; 2) limited efficiency of the computation engine due to poor data reuse and frequent weight updates; and

Received 10 March 2026; revised 10 June 2026 and 13 July 2026; accepted 27 July 2026. This article was approved by Associate Editor Davide Rossi. This work was supported in part by NSFC under Grant 92464202 and Grant U23A6007 and in part by Beijing Natural Science Foundation under Grant L2603003 and Grant L243001. *(Jiaqi Zhou and Yiqi Jing contributed equally to this work.) (Corresponding authors: Tianyu Jia; Le Ye.)*

Jiaqi Zhou, Yiqi Jing, Yiyang Sun, Siyuan He, and Tianyu Jia are with the School of Integrated Circuit, Peking University, Beijing 100871, China (e-mail: tianyuj@pku.edu.cn).

Le Ye is with the School of Integrated Circuit, Peking University, Beijing 100871, China, and also with the Advanced Institute of Information Technology, Peking University, Hangzhou 311215, China (e-mail: yele@pku.edu.cn).

Digital Object Identifier 10.1109/JSSC.2026.3718715

0018-9200 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:20:29 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

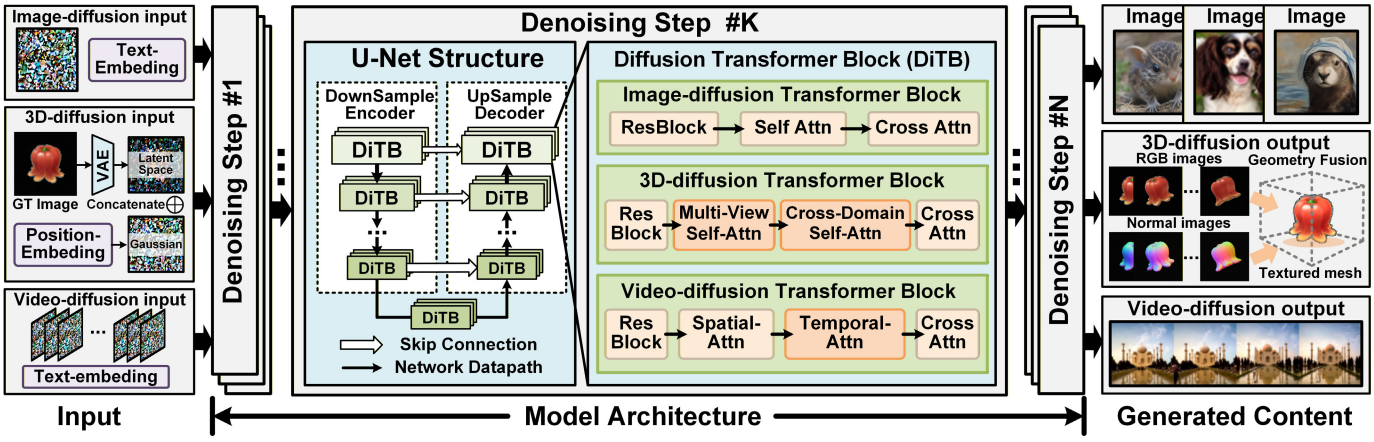


Fig. 2. Overview of the diffusion model architecture for diverse multi-task content generations, for example, for images, 3-D scenes, and videos.

3) high computational and memory overhead introduced by inter-view and inter-frame consistency operations in 3-D and video generation tasks. Consequently, our experiments show that rendering a single image on an NVIDIA RTX 3090 GPU takes over 10 s, while generating a 4-s video at six FPS on an A100 GPU server takes approximately 70 s.

In this article, we present *MTDiff* [18], an efficient diffusion accelerator tailored for multi-task content generation. At the hardware level, *MTDiff* orchestrates a dynamic bandwidth-aware memory partition scheme to enhance hardware utilization and throughput. At the circuit level, a bitline (BL)-segmented compute-in-memory (CIM) macro is developed by minimizing BL access and weight update power to improve energy efficiency. At the algorithmic level, *MTDiff* employs a hierarchical consistency management scheme to reduce redundant computational requirements. Collectively, these innovations significantly enhance the performance and energy efficiency of diffusion models for multi-task workloads. Implemented on a 22-nm technology, our chip achieves 60.81 TFLOPS/W system-level energy efficiency across multi-task diffusion models, achieving a 1.4× performance and a 1.52× energy efficiency improvement compared to prior SOTA diffusion accelerators. The contributions of this work are summarized as follows.

1) A dynamic bandwidth-aware memory partition scheme is developed to dynamically allocate bandwidth based on workload operational intensity (OI) to enhance hardware utilization and bandwidth efficiency.
2) A BL-segmented CIM macro is designed to enable concurrent computation and weight updates via local stationary units (LSUs). A reuse-aware weight update strategy further improves data reuse and adaptability.
3) A hierarchical consistency management unit is incorporated for frame-level and pixel-level optimizations to greatly reduce redundant computation and further improve computation efficiency.

II. Background and Challenges

A. Diffusion Model Architecture

As shown in Fig. 2, the diffusion model primarily consists of three components: the VAE, conditional embedding, and denoising backbone network. Guided by conditional embedding, the denoising backbone network performs multiple iterations of denoising processing to transform random Gaussian noise into the latent representation of the generated content. To understand the bottleneck of the diffusion model, we ran an open-source model, stable diffusion, on an RTX 3090 GPU. The experimental results indicate that the denoising backbone network accounts for over 99% of the computation time, making it the bottleneck for model latency. Therefore, any technique that accelerates the denoising backbone network will yield notable benefits for the entire content generation process.

For the existing diffusion algorithms, U-Net [19], [20], [21] is a widely used denoising backbone model architecture. As shown in Fig. 2, the U-Net consists of an encoder and a decoder with stacked DiT blocks (DiTB), which include residual blocks, self-attention, and cross-attention mechanisms. The encoder progressively reduces the dimensions of the feature maps while increasing the number of channels to extract high-level features. The decoder mirrors the encoder to restore the resolution and dimensions of the feature maps through interpolation upsampling. Additionally, the decoder employs skip connections to concatenate feature maps from corresponding levels of the encoder, preserving fine-grained details. In 3-D modeling tasks, a multi-view attention mechanism is incorporated to maintain consistency across different perspectives. In video generation tasks, the changes in frame content are divided into camera transitions and object movements. Therefore, a temporal–spatial attention mechanism is introduced to maintain consistency across multiple video frames, leading to additional computational demands.

We also conducted a layer-wise analysis of the denoising U-Net in the stable diffusion model to profile the computational characteristics, as shown in Fig. 3(a). The figure plots two per-layer metrics across all 30 U-Net layers grouped by feature-map resolution (Reso.1–Reso.4). The orange curve denotes the parameter size of each layer, and the blue dashed curve denotes the OI (arithmetic operations per memory access) of each layer. It is observed from the orange curve that the parameter size grows substantially at the lower resolution layers (Reso.3 and Reso.4), where feature maps are downsampled but expanded to much higher channel dimensions.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:20:29 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

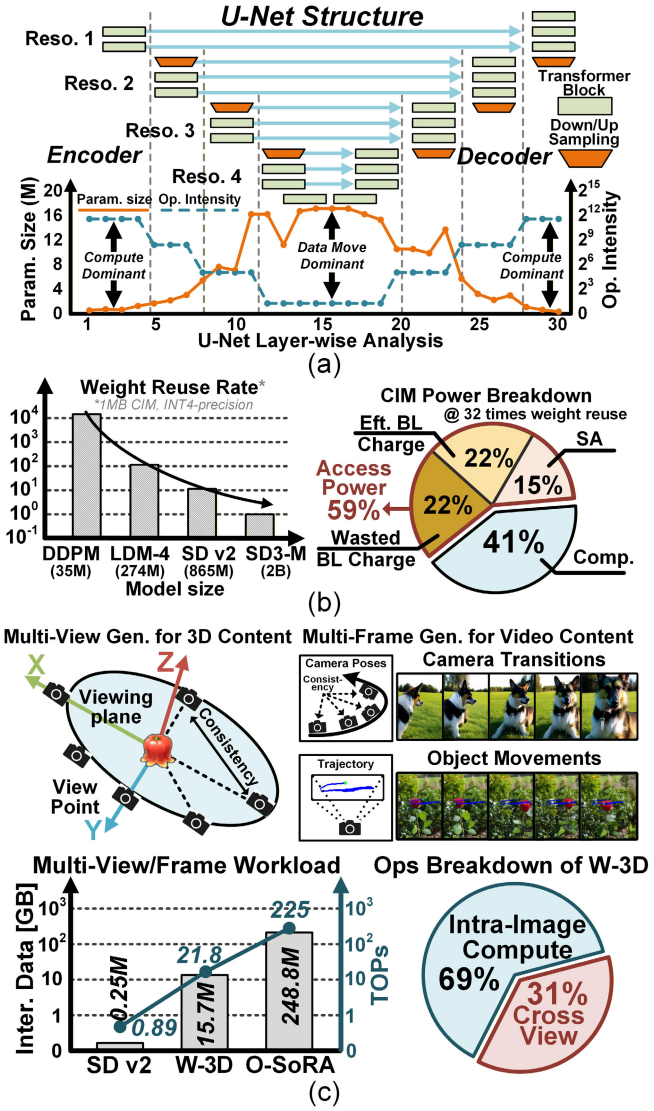


Fig. 3. Hardware deployment challenges for diffusion models. (a) Wide range of bandwidth requirement. (b) Limited efficiency of computation engine. (c) High overhead for inter-view/frame consistency.

In contrast, the higher resolution layers (Reso.1 and Reso.2) at both ends of the U-Net, which process feature maps with fewer channels, exhibit substantially higher OI. Such observations indicate a pronounced mismatch between parameter distribution and computational intensity across U-Net layers, making the conventional memory–compute mapping difficult to efficiently support all layers.

B. Deployment Challenges

Based on our analysis, we observe a few key deployment challenges for U-Net diffusion models, as explained by following.

1) Wide Range of Bandwidth Requirement: As shown in Fig. 3(a), the varying resolutions and channel dimensions across U-Net layers produce a broad spread in both per-layer data movement and OI. Lower resolution layers, for example, Reso.3 and Reso.4, transfer large data volumes with low OI, placing heavy pressure on memory bandwidth. Higher resolution layers, for example, Reso.1 and Reso.2, exhibit high OI, leading to high OI and a strong demand for compute throughput rather than bandwidth. Without the flexibility to dynamically reallocate resources to such shifting demands, hardware utilization remains severely limited. Moreover, diffusion models have approximately 10× more parameters than VAE and GAN models, increasing external memory access and further impacting performance and efficiency.

2) Limited Efficiency of Computation Engine: CIM is a promising technique with superior area and energy efficiency by performing computations directly within the memory array. As shown in Fig. 3(b), the left plot analyzes the average per-weight reuse rate for four representative diffusion models, DDPM (35M), LDM-4 (274M), SD v2 (865M), and SD3-M (2B) at INT4-precision on a total of 1 MB CIM macros. As the model size grows beyond the CIM capacity, weights need to be repeatedly reloaded from external memory, causing the reuse rate to drop sharply by over four orders of magnitude. Such frequent weight reloading disrupts computation and degrades effective performance. The pie chart also breaks down the CIM macro power at a representative 32× weight reuse based on post-layout simulations in Cadence Virtuoso with parasitics back-annotated. It is observed that 59% of CIM power is consumed by weight updates, including 15% on sense amplifiers, 22% on effective BL charging, and 22% on wasted BL charging on unselected columns. Consequently, the performance and efficiency advantages of CIM macros are substantially diminished at the system level.

3) High Overhead for Inter-View/Frame Consistency: Content generation for 3-D scenes or videos requires multi-view/frame consistency operations to ensure smooth transitions between different views or frames. To quantify this overhead, we use the PyTorch profiler to measure three representative open-source models, that is, stable diffusion v2 for image generation, Wonder3D for 3-D reconstruction, and Open-SoRA for video generation. The bar chart in Fig. 3(c) reports the per-iteration intermediate data volume and compute workload of one denoising iteration for each model. Compared to single-image generation, the per-iteration intermediate data volume for 3-D and video generation increases by 62.8× and 995×, respectively. Correspondingly, the per-iteration TOPs increase by 24.5× and 253×. The right pie chart further breaks down the per-iteration operations of Wonder3D, showing that cross-view consistency operations account for 31% of the total compute. This greatly increases computational intensity and storage overhead, while existing accelerators still lack effective mechanisms to manage these consistency operations.

III. Architecture Overview of MTDiff

To alleviate the aforementioned challenges for multi-task diffusion models, we introduced cross-stack optimizations to enhance hardware efficiency. Fig. 4 illustrates the overall architecture of our *MTDiff* accelerator, organized around three key subsystems (Features 1–3) that, respectively, target the three deployment challenges identified in Section II-B. To accommodate the bandwidth diversity and enhance hardware utilization, the memory subsystem is equipped with a dynamic memory partition scheme for flexible memory bank allocation. A 3-Mb high-density eDRAM is utilized as on-chip storage (Feature 1). To support efficient computation, a CIM-based

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:20:29 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

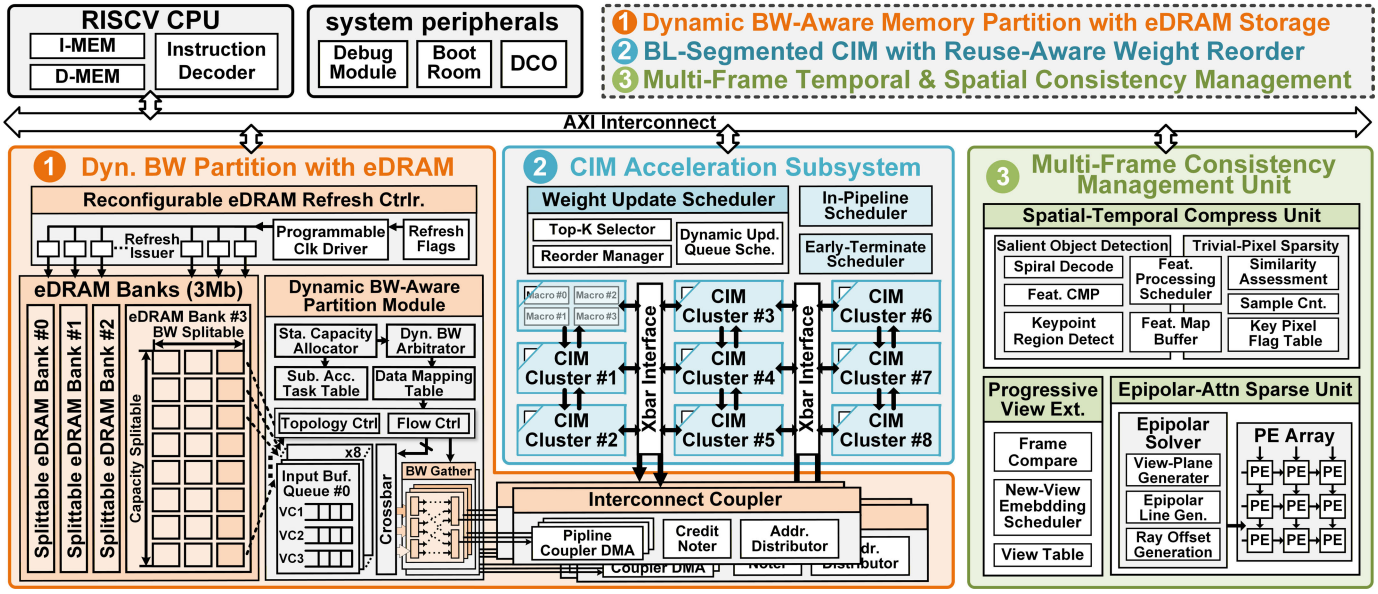


Fig. 4. Overall architecture of our *MTDiff* accelerator and the key design features.

acceleration subsystem is designed using nine CIM clusters, integrated with a weight update scheduling unit. Each cluster includes four 24 kb custom BL-segmented digital CIM macro units, specifically designed for large-scale accelerated computing tasks (Feature 2). To improve the efficiency of multi-task Diffusion, a multi-frame consistency management (MFCM) unit is employed, including hierarchical optimization at both frame and pixel levels to ensure consistency of generated results. This optimization strategy reduces redundant computations and improves the efficiency of 3-D scene reconstruction and video generation tasks (Feature3). The architecture also integrates a RISC-V CPU along with auxiliary circuits such as a debug module, boot ROM, and clock generation, supporting system debugging and operation.

A bandwidth-aware memory subsystem is designed to decouple the storage capacity from bandwidth provisioning to flexibly support both compute-intensive and bandwidth-intensive subtasks. It centers on a 3-Mb high-density eDRAM partitioned into multiple reconfigurable banks, controlled by a bandwidth-aware partition module that runs a two-phase scheduling flow combining static capacity partitioning and dynamic bandwidth arbitration. An area-efficient crossbar and interconnect coupler realize the topology and flow control, while a reconfigurable refresh controller tailors the eDRAM refresh rate to the active workload.

A CIM-based acceleration subsystem is designed as the main computation engine. Direct connections are established between macros, enabling seamless data communication. This organization is complemented by an in-pipeline module that facilitates pipelined parallelism, allowing efficient data processing and reduced latency. For effective data exchange with the storage subsystem, the CIM cluster utilizes an Xbar interface to streamline the interaction between computing and storage components. A reuse-aware weight update scheduling unit is designed to minimize the demand for frequent weight updates by carefully managing and reusing weights.

The MFCM unit utilizes a three-stage hierarchical optimization for consistency. A progressive view expansion module is designed to optimize frame-level processing through a combination of spatial and temporal techniques. For intra-frame optimization, it effectively eliminates background information that does not contribute to the desired output content. Additionally, it skips processing of trivial pixels, which only have little change between frames. For inter-frame optimization, the module employs an epipolar geometry attention mechanism to identify and prioritize the key areas of interest across consecutive frames. By focusing on these critical regions, the system minimizes redundant calculations and enhances the coherence of generated results. These optimizations significantly reduce both intra-frame and inter-frame computational redundancy.

IV. Bandwidth-Aware Memory Partition

The bandwidth-aware memory partition scheme addresses the imbalanced OI across diffusion model layers by dynamically allocating on-chip memory and compute resources. The core idea is to treat the four eDRAM banks (each with 3×8 arrays) and nine CIM clusters (each containing four CIM macros) as configurable resource pools. By grouping different numbers of eDRAM arrays and CIM clusters into task-specific compute groups, both the available storage bandwidth and compute throughput can be tuned to match each subtask's OI. Specifically, bandwidth-bound subtasks are assigned fewer CIM clusters but more eDRAM arrays to increase on-chip cache capacity and reduce redundant data movement overhead, while compute-intensive subtasks with high data reuse are allocated more CIM clusters with fewer memory arrays. This decoupling of storage capacity from bandwidth provisioning enables efficient execution of heterogeneous workloads simultaneously.

A. Workflow Overview

In conventional fixed hardware mapping schemes, the imbalanced inter-layer demands significantly reduce hardware

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:20:29 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

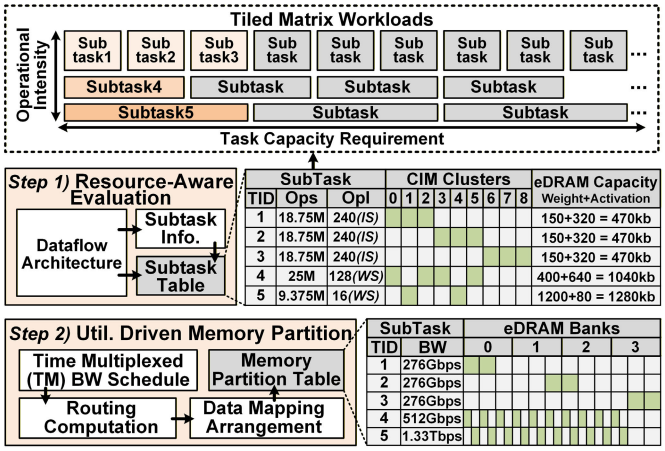


Fig. 5. Our two-stage bandwidth-aware memory partition scheme.

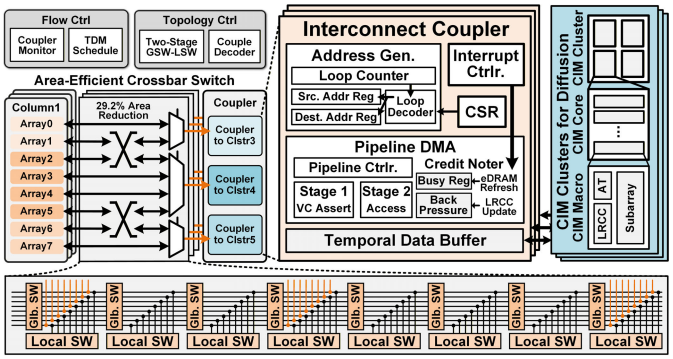


Fig. 6. Implementation of interconnect coupler and flow control circuits.

utilization. In this work, we present a bandwidth-aware partition based on the dynamic bandwidth requirements of different subtasks, allowing for flexible hardware resource allocation based on specific workload demands. As shown in Fig. 5, we employ a two-phase partitioning strategy for the dynamic memory resource allocation. Initially, the computational and storage demands of each subtasks from the diffusion workload are evaluated. Based on the bandwidth demand characteristics, the data is dynamically arranged within the storage arrays, for example, allocate more storage capacity for high bandwidth provision. The topology and flow control circuits manage the data movement according to subtask allocation. The workflow details are explained as follows.

Stage 1 (Resource-Aware Bandwidth Evaluation): This stage is performed offline at compile time by a host-side compiler before deployment. The diffusion model is divided into subtasks, that is, the matrix loads in different diffusion layers. The compiler determines the number of CIM clusters and required eDRAM capacity for each subtask based on its OI. The number of assigned CIM clusters is determined by comparing each subtask's OI against the system ridge point $OI^* = C_{\text{cluster}}/BW_{\text{cluster}}$. Subtasks with $OI > OI^*$ receive more CIM clusters, while those with $OI < OI^*$ receive more eDRAM arrays. To further maximize hardware utilization, subtasks with complementary OI profiles are co-scheduled, ensuring CIM compute resources and eDRAM bandwidth are simultaneously saturated. The evaluation results are recorded in a resource-aware subtask table, which details the CIM computational hardware and the corresponding storage space required for each subtask. This table is loaded onto the chip prior to inference as part of the deployment configuration. For example, subtasks 1–3 are compute-intensive, occupying three CIM clusters and requiring 470 KB of storage space. In contrast, subtask 4 has a more balanced demand for computational and storage resources, needing four CIM clusters and 1040 KB of storage space. Subtask 5 is storage-intensive, requiring only two CIM clusters but needing 1280 KB storage, along with high-bandwidth data transfer capabilities.

Stage 2 (Utilization-Driven Memory Partition): This stage is executed on-chip during runtime by the dynamic BW-aware partition module shown in Fig. 4. Driven by the resource-aware subtask table generated in Stage 1, the dynamic BW arbitrator inside the partition module decides which eDRAM bank holds each data block by jointly considering its bandwidth demand and the CIM compute resources allocated to the corresponding subtask. For instance, subtasks 1–3 require only two columns of eDRAM storage and corresponding bandwidth to meet their computational demand. In contrast, subtasks 4 and 5, which require higher bandwidth during computation, utilize a finer-grained storage partition scheme. By distributing data across different arrays, this approach decouples storage capacity from available bandwidth to provide higher effective peak bandwidth. A time-division multiplexing (TDM) bandwidth scheduler further refines the bandwidth allocation. Since the total data transferred by each subtask equals the allocated bandwidth multiplied by its active time window, the TDM scheduler adjusts time-window durations to balance bandwidth utilization using a greedy policy. Subtasks with the largest bandwidth demand are prioritized and assigned longer time windows in each scheduling cycle, iterating until all subtasks' bandwidth demands are satisfied. The memory partition table records the assigned eDRAM array indices, address mappings, CIM cluster IDs, and bandwidth time-slot allocations. It drives three downstream components: the DMA for data routing, the TDM scheduler for sequencing eDRAM accesses, and the eDRAM refresh controller for selectively disabling refresh on inactive arrays. For example, the bandwidth values in Fig. 5 (276 Gb/s–1.33 Tb/s) represent subtask demands. The architecture satisfies these demands through spatial allocation of eDRAM banks and temporal multiplexing. High-bandwidth subtasks are assigned more arrays to aggregate bandwidth spatially, while time-division scheduling orchestrates transfers to meet peak requirements.

B. Implementation Details

Based on the above bandwidth-aware partition scheme, we designed corresponding topology and flow control circuits to facilitate data interaction between the storage arrays and CIM clusters, as shown in Fig. 6. The area-efficient crossbar architecture employs a two-stage hierarchical switching mechanism. At the first stage, local switches (local SW) are distributed to each eDRAM array within a column, providing array-level access control. At the second stage, global SWs (Glb. SWs) aggregate data from multiple local SWs and establish connections to the CIM cluster couplers. For example, by disabling the third Glb. SW and sequentially

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:20:29 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

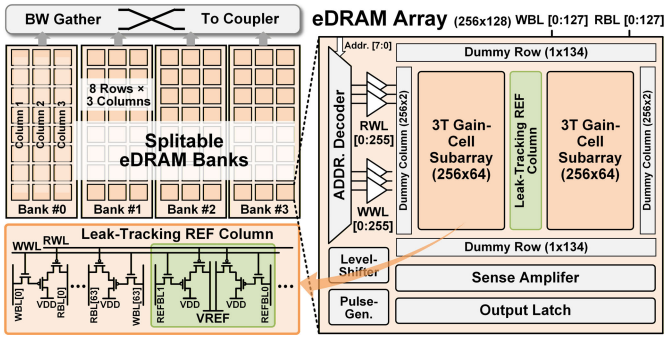


Fig. 7. Design details of on-chip splitable eDRAM storage.

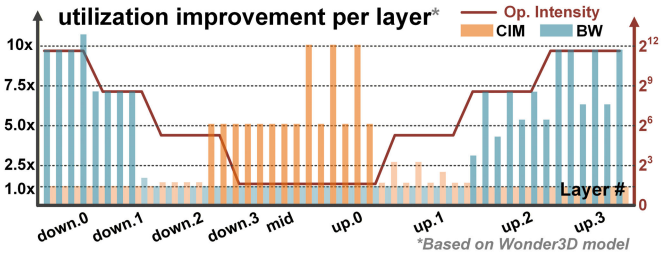


Fig. 8. Evaluation for the dynamic BW-aware memory partitioning.

activating the first and second local SW in a time-division manner, Array 0 and Array 1 can be allocated to Cluster 3. Given the deterministic nature of diffusion data flows, we employed a compact crossbar design. We utilize predefined data flow patterns to simplify the circuit design to a two-stage global and local switching mechanism for the crossbar. Instead of using synthesized multiplexer-based selectors, custom hand-crafted transmission gate-based SWs are implemented, combined with custom switch control encoding and hardware decoders for topology control. This design meets the flexibility demands of data transmission while reducing switch circuit area overhead by 29.2% compared to a fully connected synthesized multiplexer-based crossbar. This comprises 12.9% from topology optimization and 18.9% from the custom transmission-gate implementation.

The data interactions are facilitated through an interconnect coupler, which includes an address generator, pipelined DMA, interrupt control logic, and temporary data buffers. Under the control of the TDM schedule unit, it manages the data transfers between the eDRAM and the CIM clusters. The address generator, governed by the CSR, utilizes a loop controller to generate source and destination addresses. The pipelined DMA consists of two stages: the first stage determines and reserves virtual channels, while the second stage executes data transfer operations. A credit noter handles interrupt signals, managing eDRAM refresh operations and data backpressure caused by updating the LSU in CIMs.

During the implementation, we utilize eDRAM to enhance on-chip storage capacity and efficiency. As shown in Fig. 7, the storage is split into four groups, each consisting of a 3×8 array, with each array containing 256×128 3T gain-cell structures. Dummy row and column cells are employed to ensure yield reliability. Additionally, leakage tracking columns are incorporated to extend data retention time. The 3T gain-cell eDRAM exhibits a worst-case retention time of 2 μs at 0.6 V and 85 °C, extending to 3 μs at 25 °C. The reconfigurable refresh control unit adaptively adjusts the refresh rate (333 kHz at typical conditions) and selectively disables refresh for inactive data blocks. During bandwidth-intensive phases, refresh operations consume approximately 8.5% of peak memory bandwidth. Diffusion's iterative denoising (20–50 steps) provides algorithmic error resilience through high information redundancy, where bit-flip errors are naturally corrected as the network treats anomalies as removable noise. PVT characterization (from −40 °C to 125 °C, ±10% voltage) shows less than 0.5% FID degradation without ECC. The 3 Mb eDRAM occupies 1.38 mm^2 (16.8% of chip area), achieving a 32% area reduction compared to foundry SRAM of equivalent capacity. These splitable eDRAM banks connect the CIM clusters via an area-efficient crossbar switch to couplers. These couplers, each corresponding to a CIM cluster, serve as data interfaces for CIM data exchange control. Other types of memory, such as SRAM, are also applicable to this architecture scheme.

The fixed uniform resource allocation statically assigns equal shares of CIM clusters, eDRAM capacity, and bandwidth time slots to each subtask, regardless of actual demands. This results in compute-intensive layers being starved of CIM resources, while memory-intensive layers are left with idle clusters. Our bandwidth-aware partition scheme dynamically adjusts the allocation of bandwidth and computational resources according to the specific needs of various tasks. Consequently, different types of workloads can be executed in parallel within the same system efficiently. We evaluate the design benefits using practical use cases from the Wonder3D model [22] for the 3-D model generation task, as shown in Fig. 8. It reveals significant layer-wise heterogeneity in OI. Our partitioning dynamically allocates more eDRAM array to memory-bound layers to eliminate CIM idle time, while allocating more CIM to compute-bound layers to match data throughput. It is observed that our scheme achieves 2.68× average CIM utilization improvement and 4.73× average BW utilization improvement compared to a fixed uniform resource allocation scheme under the same total compute, bandwidth, and storage budget, showing our flexible resource scheduling strategy significantly enhances hardware utilization efficiency.

V. Digital CIM Macro

A. BL-Segmented CIM Macro

Our acceleration subsystem comprises nine CIM clusters and a weight update scheduling unit. Each cluster contains four CIM macro units, which are directly interconnected through an internal aggregation unit to support pipelined parallelism. To reduce the access power of CIM, an efficient BL-segmented CIM macro is developed. As shown in Fig. 9, each CIM macro incorporates six 128×32 MAC arrays based on 6T SRAM. Each MAC array is composed of 4×4 subarrays, with each subarray containing four weight banks and a lookup table (LUT)-bypass adder tree. Each weight bank includes 4×16 6T SRAM cells and an LSU. This array employs a hierarchical storage structure and an LUT-based bypass adder tree, as detailed below.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:20:29 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

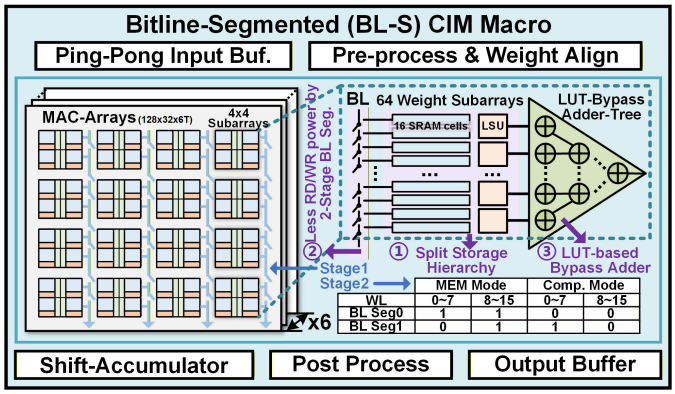


Fig. 9. Design details of BL-segmented CIM macro.

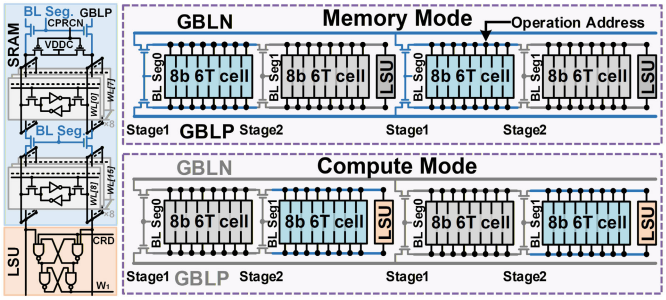


Fig. 10. Storage hierarchy and BL segmentation in CIM macro.

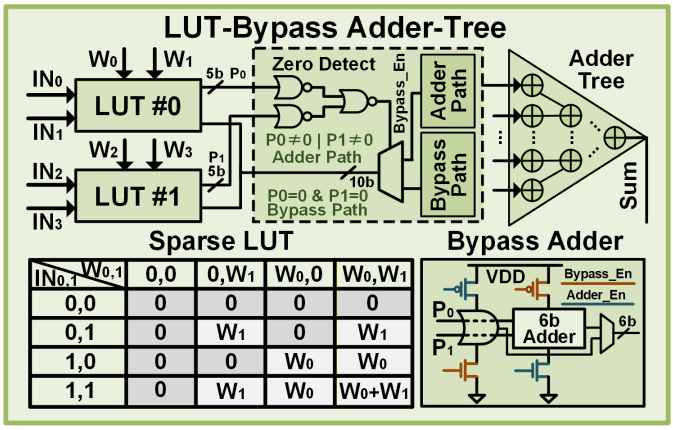


Fig. 11. LUT-based bypass adder tree in CIM.

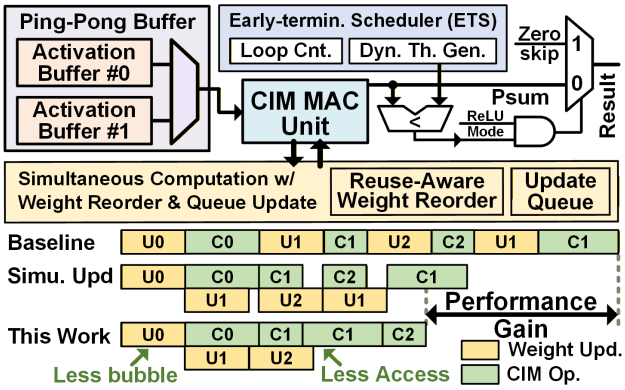


Fig. 12. Design overview of the weight update scheduler.

As shown in Fig. 10, a hierarchical storage structure is employed, comprising SRAM banks to store multiple weight sets and LSUs using latches. The LSU is an area-efficient NAND latch. Each 16-bit SRAM cell is paired with a 1-bit LSU to temporarily hold the result of the 16-to-1 wordline selection. The LSU serves as a computation-side buffer that decouples MAC operations from SRAM storage. During computation, weights are preloaded from SRAM into the LSU, which then directly supplies the MAC units. This decoupling frees the shared SRAM BLs, enabling concurrent weight updates without interrupting ongoing computations. Compared to traditional 8T read–write separate SRAM implementations for simultaneous computation and weight updates [23], [24], this design offers enhanced area efficiency by sharing the LSUs across multiple SRAM matrices. A two-stage BL segmentation is implemented, which segments BLs based on the accessed address to minimize BL load. The first stage involves BL segmentation at the weight bank level, while the second stage further segments BLs within the weight bank. During weight updates, the CIM enters storage mode, activating the first-stage BL SWs based on the operation address. Within the targeted weight bank, the second-stage BL segmentation is executed as specified. During LSU updates, the CIM switches to computation mode, eliminating the need for global bitlines. The second-stage switches activate only the addressed bitline segment within the weight bank.

To further mitigate frequent low-bit toggling during computation, an LUT-based bypass adder tree is employed in our CIM. As shown in Fig. 11, a four-depth sparse LUT fuses the multipliers and the first adder tree stage, indexed by two 1-bit activation inputs (2-bit address, four entries). Each LUT stores the pre-computed sum $W_0 \times IN_0 + W_1 \times IN_1$ for the corresponding two 4-bit weights and activation input bits. For W4A8 operations, activations are bit-serially fed across eight cycles, with cycle i processing the ith bit from each activation. Partial products are bit-shifted and accumulated through subsequent adder tree stages. For BF16 operations, floating-point values are first converted to 12-bit integers via exponent alignment, then processed bit-serially across 12 cycles. The second stage features an adder with bypass detection to perform zero-skipping, further reducing ineffective dynamic toggling.

B. Reuse-Aware Weight Update Scheduler

We also adopt a reuse-aware weight update scheduler to facilitate weight updates concurrent with computation. As shown in Fig. 12, the weight updates in traditional CIM macros occupy global BLs and interrupt computation that requires local BLs. This interruption degrades peak performance for diffusion models due to frequent weight updates. The CIM macro using 8T read–write decoupled SRAM cells for simultaneous computation and update has degraded area efficiency. In this work, we adopt simultaneous computation with weight reorder and queue update techniques. These techniques effectively mitigate pipeline bubbles and further reduce the memory access overhead of weight updating, thereby improving overall

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:20:29 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

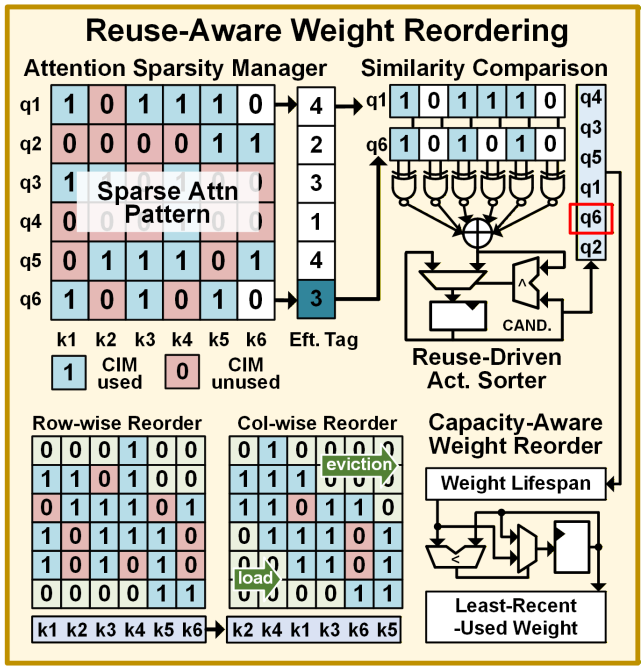


Fig. 13. Workflow of the reuse-aware weight reordering in the scheduler.

execution efficiency of CIM units. Our scheduler incorporates a ping-pong buffer mechanism that enables concurrent read and write operations, allowing new weights to be loaded while computation proceeds. Additionally, an early termination scheduler detects ReLU zero outputs and terminates unnecessary downstream computations, reducing redundant operations and improving energy efficiency.

Fig. 13 illustrates the detailed workflow and implementation of the reuse-aware weight reordering, which consists of an attention sparsity manager, a similarity comparison module, and a capacity-aware weight reordering unit. The attention sparsity manager stores the initial sparse attention pattern. For multi-view tasks, this pattern is derived from the epipolar-geometry-based mask generated by the MFCM unit (detailed in Section VI-A Stage 3). For single-view tasks, it initializes with a Top-K pattern and updates the attention pattern based on the runtime token pruning information. Then, the manager iteratively computes row-wise and column-wise sums of the sparse attention pattern. If the sums are larger than a task-specific predefined threshold, the corresponding query and key vectors of q and k are selected for global-like attention. Their indices are stored in the global attention FIFO. The local attention sorter reorders the rest attention as a local-like pattern. The reordering is performed globally across the entire attention matrix. This global scope is enabled by comparing only binary attention map masks rather than full-precision q/k vectors, which maintains low hardware complexity while maximizing q/k reuse and avoiding redundant weight loading. In the similarity comparison module, the similarity-based q-sorter row-wisely reorders q and determines the input feeding sequence. Then, the difference-based k-sorter column-wisely reorders k and determines the weight writing sequence.

In this example of Fig. 13, q6 has the greatest similarity to q1, so it is selected as the next input q vector, and the k vectors already in CIM can be reused continually. Subsequently, the same reordering strategy is applied to the k vectors. The vector indices for input feeding and weight writing are stored in the local-q and local-k FIFOs. As a result, the sparse attention pattern is reorganized into a near-diagonal pattern, enabling deferred on-chip loading of unused q and k vectors while allowing the early eviction of fully utilized vectors. Meanwhile, the data retention time within the CIM array is effectively extended, thereby reducing frequent data refresh and update overhead. The scheduler's outputs are configurations for the CIM macros under the reshaped attention sparsity. They are produced by the reshaped attention generator based on the sequence in the three FIFOs for global, local-q, and local-k attention. The configurations consist of two parts: the first part is to control the input buffer to feed tokens for q and k generation. The second part is to control each CIM macro's workload assignment.

Based on our analysis, the BL segmentation and LUT bypass adder tree and weight update scheduling unit provide energy efficiency improvements of 27%, 11%, and 29%, respectively. The sparsity-aware reordering effectiveness correlates with attention sparsity across tasks. For instance, the image generation with 48% sparsity achieves 2.1× attention weight (q/k vectors stored in CIM) reuse. The 3-D reconstruction with 68% sparsity achieves 2.8×, and video generation with 76% sparsity achieves 3.2× improvements. Overall, the BL-segmented CIM macro achieves a 1.31× performance improvement and a 54% reduction in energy consumption.

VI. Multi-Frame Consistency Management

A. Workflow Overview

In 3-D reconstruction and video generation tasks, additional computation is required to maintain visual consistency across different views/frames. Due to the inherent geometric correlations among views/frames, varying degrees of computational redundancy are introduced. To alleviate the consistency computations, we implemented a three-stage consistency management, as illustrated in Fig. 14 and explained as follows.

Stage 1 (Progressive View Extension): In the diffusion-based 3-D reconstruction, images from multiple viewpoints are processed in parallel. Based on our analysis, it is observed that the denoising outputs across different views are highly similar during the initial timesteps. However, after several iterations, the results gradually exhibit view-specific features. Therefore, we develop a progressive view expansion scheme, where only the front view is processed during the early timesteps. For example, at the fifth timestep, a new back view is introduced by incorporating its corresponding view embedding into the computation. Subsequently, the remaining views are progressively activated until all six canonical views are fully reconstructed. By gradually expanding the number of active views, redundant computations in the early timesteps are effectively avoided. The timing for introducing new view embeddings is adaptively determined by evaluating the discrepancy between candidate and reference views, thereby ensuring both computational efficiency and reconstruction fidelity.

Stage 2 (Background and Trivial Pixel Skipping): During the content generation, two types of non-critical pixel redundancy are commonly observed: 1) static background pixels and

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:20:29 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

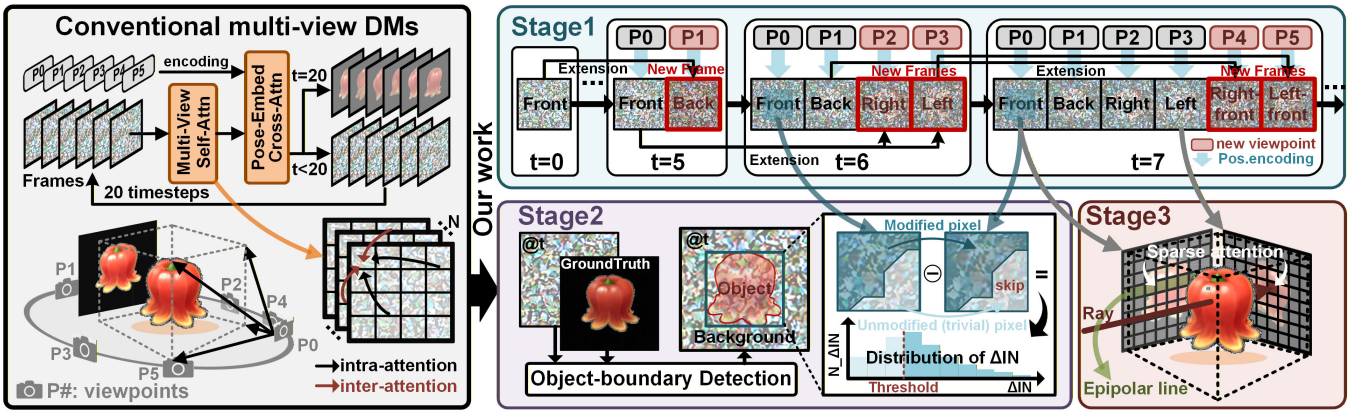


Fig. 14. Computational analysis of multi-view consistency requirement and our three-stage optimizations.

2) dynamic denoising pixels. For 3-D reconstruction tasks, the target object is typically in the center area, with the surrounding regions padded using uniform black or white background pixels. Since these background pixels do not contribute to the reconstruction objective, they can be safely excluded from computation, thereby reducing both computational complexity and memory consumption. Moreover, during the iterative Diffusion denoising process, it is observed that not all pixels undergo uniform refinement at each timestep. The model selectively focuses on salient regions through attention mechanisms, while trivial pixels exhibit low variation across timesteps. Therefore, our management dynamically identifies and only updates the actively denoised pixel locations at each timestep to further avoid unnecessary computations.

Stage 3 (Epipolar-Attention Sparse Processing): During the attention stage, different viewpoints exhibit inherent geometric correlations in the spatial domain. This insight motivates us to exploit such geometric priors to reduce inter-frame computation, thereby mitigating redundancy in multi-view/frame processing. For example, in the inter-frame processing stage, each pixel in the image plane of camera P3 corresponds to a projection ray in 3-D space, which maps onto an epipolar line in the perspective of camera P0. Pixels located along this epipolar line in P0's image plane contribute most significantly to the interaction with the corresponding pixel in P3's view. Accordingly, the attention computation prioritizes these geometrically relevant pixels, enabling more efficient and targeted cross-view feature aggregation. Specifically, off-epipolar pixels are marked as non-critical through a binary mask generated by the Epipolar Solver, which is fed to the attention sparsity manager as mentioned in Section V-B to guide CIM sparse attention computation.

B. Implementation Details

To support the aforementioned three-stage processing, we implement corresponding hardware for each stage, as shown in Fig. 15. In Stage 1, we integrate a dedicated frame comparison module to determine whether a new-view embedding should be introduced into the denoising process for generating an additional viewpoint. The frame comparison module evaluates the reference frame against the frame at the current timestep. Specifically, the RGBA value of each pixel in the reference

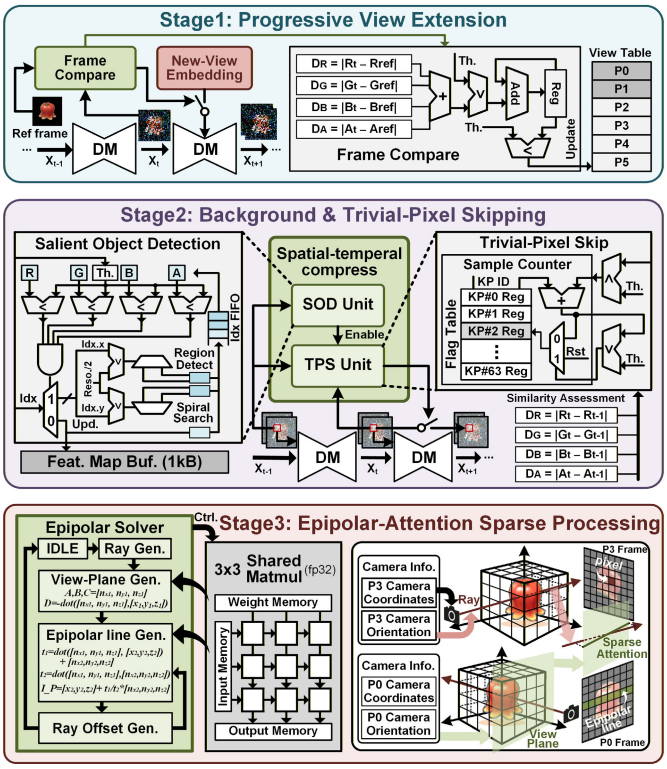


Fig. 15. Implementation of the three stages consistency management.

frame is subtracted from that of the corresponding pixel in the current frame, and the difference is compared with a predefined threshold. When the number of pixels whose differences exceed the threshold satisfies a preset criterion, the frame comparison module updates the view table to indicate that a new viewpoint should be activated and incorporated into subsequent denoising iterations.

In Stage 2, a spatial–temporal compression module, consisting of a salient object detection (SOD) unit and a trivial-pixel skip (TPS) unit, is employed to determine whether each pixel should participate in computation. Since the background is filled with a uniform color, it is easy for the SOD unit to identify background pixels by directly comparing their RGBA values against predefined values. Since the target

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:20:29 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

object in 3-D reconstruction tasks is typically centered in the image, the SOD unit adopts a spiral scanning strategy starting from the image center and expanding outward in a spiral pattern. The scanning terminates when a complete spiral loop consists entirely of background pixels, indicating that the object boundary has been fully enclosed, enabling rapid boundary detection of the target object with minimal pixel comparisons. The TPS unit compares the RGBA values of corresponding pixels between two adjacent timesteps and records the results in a flag table when the variation falls below a predefined threshold. If a pixel remains unchanged, it is classified as a trivial pixel and skipped in the computations.

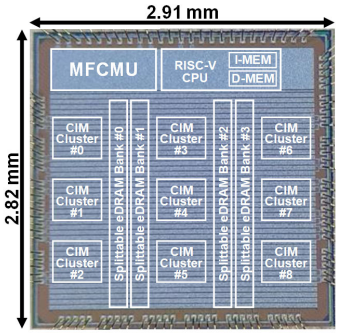
In Stage 3, an Epipolar Solver is designed to enable attention sparsification by computing epipolar geometry transformations. The solver employs a 3 $\times$ 3 FP32 PE array to perform the 3 $\times$ 3 matrix multiplications required for camera projection and plane intersection calculations in epipolar geometry. The PE array employs an output-stationary dataflow, enabling multiple dot products to be computed in parallel. The dataflow of the solver consists of four steps: 1) ray generation; 2) view-plane generation; 3) epipolar line generation; and 4) ray-offset generation. Leveraging camera information, including camera coordinates and orientation: 1) generates the projection rays by querying precomputed direction vectors; 2) generates the view plane corresponding to the given camera viewpoint, with the required dot-product operations executed on the PE array; and 3) fetches the projection ray and the view plane from two respective camera positions and computes their intersection via the PE array to establish the epipolar geometry relationship. For example, the ray from P3 and the view plane from P0. They are jointly processed to establish the epipolar geometry relationship. This geometric constraint sparsifies attention by restricting interactions to pixels along the corresponding epipolar line. To reduce the computationally intensive processing in 3), 4) enables efficient epipolar line generation through offset calculation for camera positions with simple geometric relationships, which is directly computed on the RISC-V processor without invoking the PE array.

Evaluated on the Wonder3D model, the three optimization stages provide speedups of 1.38×, 1.88×, and 1.43×, respectively, measured incrementally as each stage is added. MFCM also offers a reduction of 61% memory access and 68% power consumption relative to the baseline implementation without frame-level or pixel-level pruning. The MFCM does not alter the model's mathematical formulation, while it selectively prunes negligible computations. MFCM introduces less than 0.3% FID degradation, which is masked by the 1.2%–1.5% degradation from W4A8 quantization, resulting in negligible impact on generation quality.

VII. Experimental Results

A. Chip Micrograph and Specification

Fig. 16 presents the micrograph of the *MTDiff* chip and specification. It is fabricated in a 22-nm CMOS technology and occupies a die area of 2.82 $\times$ 2.91 mm^2. During testing, the host PC sends assembly code to the RISC-V CPU on the test chip via the JTAG protocol. The RISC-V CPU controls the accelerator subsystem through a predefined instruction set



Technology	TSMC 22nm		
Chip area	2.91mm×2.82mm		
CIM Macro area	0.149mm×0.168mm×36		
CIM Size	864 kb		
Memory Size	3 Mb eDRAM + 64kB SRAM		
Weight prec.	INT4/8/16, FP8, BF16		
Activation prec.	INT4/8/16, FP8, BF16		
Voltage	0.6 - 0.9 V		
Frequency	180 - 478 MHz		
CIM Power	33.7 - 127.6 mW		
System Power	73.8 - 211.1 mW		
Performance[*1]	6.79 - 9.71 T(FL)OPS (Hybrid-precision)		
CIM Macro[*2] Energy Efficiency	76.69 - 132.97 TFLOPS/W (BF16/BF16)		
System[*2] Energy Efficiency	49.74 - 60.81 TFLOPS/W (Hybrid-Precision)		
Task	Image	3D	Video
Throughput	3.82 iter/s	3.70 iter/s	0.02 iter/s
Energy per DM iteration	29.23 mJ[3] (Single-Frame)	39.72 mJ[4] (12-Frames)	9.07 J (25-Frames)

1. Measured @ 0.9V, 478MHz. 2. Measured @ 0.6V, 160MHz.
3. Image Resolution: 64x64 4. Image Resolution: 32x32

Fig. 16. Die photograph of fabricated chip and detailed specifications.

TABLE I
Area and Power Breakdown of MTDiff Chip

Component		Area(mm^2)	Power(mW)*
RISC-V CPU		0.45	9.5
MFCMU	Frame Compare	0.07	0.8
	ST Compress	0.11	1.6
	Epipolar Solver	0.23	2.5
CIM Cluster	4× CIM Macro	0.10	3.7
	Peripheral Logic	0.12	
3Mb eDRAM		1.38	19.7
Others		1.88	6.0
Total Core Area (22nm)		6.10	73.8

*@ 0.6V, 160MHz, Stable Diffusion v1.5

to collect test data. *MTDiff* chip contains 864 kb CIM and 3 Mb eDRAM. It supports a wide range of integer precisions (INT4/8/16) for both activations and weights for mixed-precision matrix multiplication operations. For quantization, FP8/BF16 scaling factors are used for activation and weights. The floating-point operations are tightly coupled with the CIM macros: before feeding data into the CIM arrays, the exponents of floating-point values are aligned, and the mantissas are converted to 12-bit integers, enabling the CIM macros to perform computations with negligible precision loss while maintaining hardware efficiency. The chip operates at 180–478 MHz with a supply voltage range of 0.6–0.9 V. Evaluation results are reported under two representative operating conditions: "Best Efficiency" is measured at 180 MHz and 0.6 V, and "Best Performance" is measured at 478 MHz and 0.9 V. This distinction highlights the tradeoff between low-voltage operation for highest TOPS/W and high-frequency operation for peak throughput. We implement segmented on-chip power domains in the *MTDiff* chip to enable fine-grained power breakdown. Under the "Best Efficiency" condition, when running the stable diffusion v1.5 image generation task, the CIM, which delivers over 90% of the total computational performance, consumes only 33.7 mW, accounting for 45.7% of the overall system power. The detailed area and power breakdown is shown in Table I.

We conduct a comprehensive evaluation of *MTDiff* across three representative diffusion-based generative tasks, including

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:20:29 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

TABLE II
COMPARISON TABLE WITH PRIOR SOTA DESIGNS

	ISSCC'24 [24]	VLSI'24 [29]	ISSCC'23 [30]	ISSCC'23 [31]	This Work		
Technology [nm]	28	22	28	28	22		
Task	Diffusion	Diffusion	Transformer	CNN	Diffusion		
Content Generation	Image	Image	\	\	Image	3D	Video
Data Precision	INT10/16, FP/BF16	FP16	INT8/16	INT4/8, FP/BF16	INT4/8/12, FP8, BF16		
Chip Area [mm^2]	3.67	3.7	14.36	4.54	8.2		
Supply Voltage [V]	0.6 - 1.0	0.6 - 1.0	0.6 - 1.0	0.397 - 0.9	0.6 - 0.9		
Frequency [MHz]	50 - 540	120 - 540	85 - 275	10 - 400	180 - 478		
System Power [mW]	8.268 - 171.0	60 - 279	29.83 - 152.75	0.87 - 74.9	73.8 - 211.1		
Memory Size	200kB SRAM	272kB SRAM	192kB SRAM	512kB SRAM	3Mb eDRAM + 64kB SRAM		
On-Chip BW[*1]	414.72	518.4	1126.4	819.2	2202.6		
BW Management	\	\	Static Model-aware	\	Dynamic BW-aware		
Frame Optimizations	Pixel-level (Single View)	Pixel-level (Single View)	\	\	Frame-level & Pixel-level (Multi-View)		
MAC Size	288kb + 64kb DCIM	432 PE	1024kb DCIM	64kb DCIM	864kb DCIM		
CIM Macro Efficiency [TFLOPS/W]	80.3 (FP16)	\	16.8 - 83.5 (INT16)	17.2 - 91.3 (FP16)	76.69 - 132.97 (BF16)[*2]		
Effective System Efficiency [TFLOPS/W][*3]	40.06 (BF16)	1.57 - 48.95	10.7 - 53.5 (INT16)	2.83 - 15.0 (FP16)	60.81	54.91	49.74
Macro FoM[*4]	65.1	\	4.09	30.74	83.63		
System FoM[*5]	3.69	13.2	35.1	1.54	50.44		

1. Based on maximum reported frequency. 2. Measured @180MHz, 0.6V.
3. Considering CIM utilization based on available on-chip memory and bandwidth.
4. Macro FoM = Macro Energy Efficiency × Macro Area Efficiency, evaluated @ 0.75V, area is scaled to 22nm.
5. System FoM = System Energy Efficiency × On-chip Bandwidth, evaluated based on reported peak efficiency point.

Model		SD-v1.5	Wonder-3D	SVD
Task		2D Image	3D Modeling	3D/4D Video
Dataset		LION-5B (64*64)	Objaversr (32*32)	Panda-70M
Parameters (MB)		870	910	1504
Precision	Conv	INT8	INT8	INT8
	Attention	BF16	BF16	BF16
Baseline Accuracy		17.77(FID↓)	0.119(LPIPS↓)	136.87(FVD↓)
Chip Accuracy		18.69	0.116	141.13
Execution Time(s)[*1]		13.092[*3]	5.404	1392.641
Macro Energy Efficiency (TFLOPS/W)[*2]		132.97	101.97	76.69
System Energy Efficiency (TFLOPS/W)[*2]		60.81	54.91	49.74

1. Measured @478MHz, 0.9V; 2. Measured @180MHz, 0.6V;
3. 18.378s reported in [10], off-chip memory access is excluded.

Fig. 17. Evaluation result on content generation benchmarks.

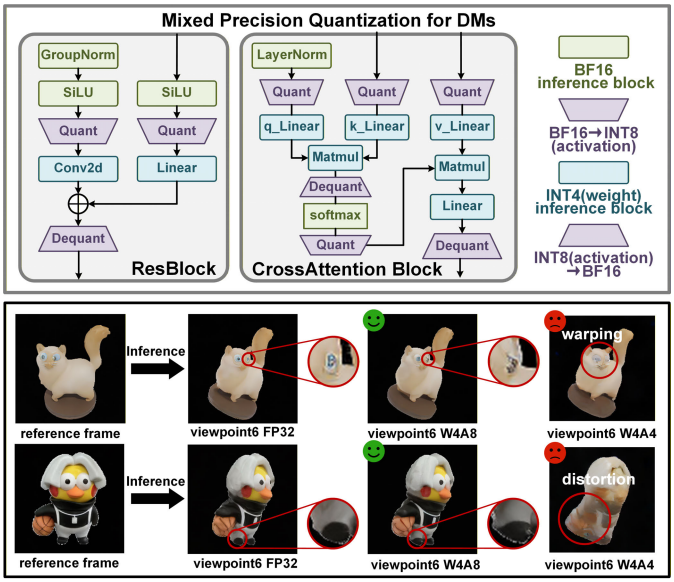


Fig. 18. Visual results of our mixed precision quantization for DMs.

image generation, 3-D reconstruction, and video generation, as shown in Fig. 17. In the image generation task, we evaluate *MTDiff* using stable diffusion v1.5 [8] and adopt LAION-5B [25] as the validation set. *MTDiff* generates 64 × 64 images in 13.092 s with an equivalent system energy efficiency of 60.81 TFLOPS/W. For the 3-D scene reconstruction task, evaluation is conducted on Wonder-3D [22] with the Objaverse [26] dataset for validation. *MTDiff* reconstructs 32 × 32 targets in 5.404 s with equivalent system energy efficiency of 54.91 TFLOPS/W. In the video generation task, *MTDiff* is evaluated using stable video diffusion [27] and validated on Panda-70M [28]. It generates a 720P high-definition (HD) video sequence in 1393 s, achieving an equivalent system energy efficiency of 49.74 TFLOPS/W.

B. Evaluation Results

1) Model Quantization: Fig. 18 illustrates the quantization strategy tailored for Diffusion models and the generated image quality. The baseline model adopts uniform FP32 precision, resulting in substantial bandwidth and computational demands for hardware. To balance generation quality and efficiency, we employ a mixed-precision quantization scheme. In residual blocks, precision-sensitive GroupNorm and SiLU layers adopt BF16 precision, while convolutional and linear layers are surrounded by quantization/dequantization modules. In cross-attention blocks, LayerNorm and Softmax remain in BF16 precision, whereas QKV projections and matrix multiplications are quantized. Weights are quantized to INT4 due to their uniform distribution, while activations are quantized to INT8 to alleviate outlier-induced accuracy loss. A post-training

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:20:29 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

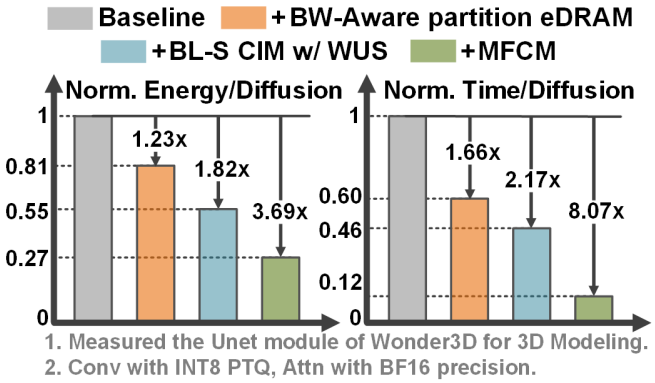


Fig. 19. Technique breakdown analysis on *MTDiff*.

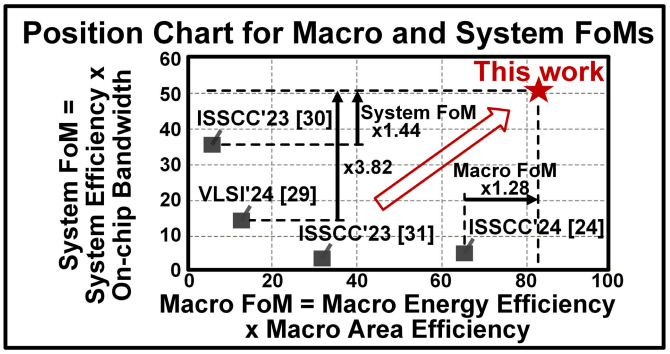


Fig. 20. Position chart of macro and system FoMs.

quantization strategy is employed, with per-layer quantization and dequantization scales calibrated offline. In the quant module, BF16 activations are scaled and truncated to INT8 for low-cost CIM-based linear computation. In the dequant module, INT8 outputs are cast to BF16 and rescaled before nonlinear operations.

We evaluate the above quantization strategy on Wonder3D for illustration and conduct a comparative study across different precision configurations. The results demonstrate that the adopted W4A8 mixed-precision scheme effectively preserves fine-grained 3-D object details with FID degradation of less than 1%, and exhibits strong generalization across diverse object inputs. In contrast, the more aggressive W4A4 quantization leads to noticeable quality degradation and shows higher sensitivity to the choice of quantization calibration dataset.

2) Improvement Breakdown: Fig. 19 provides a detailed analysis of *MTDiff*, illustrating the energy savings and performance enhancements achieved with *MTDiff*'s features. We still utilize Wonder3D as the illustration model. We adopt a uniform INT8 post-training quantization scheme for convolutional operations, while retaining BF16 precision for attention operations, which is commonly adopted in industry practice. As the baseline, a PE-based accelerator with equivalent compute capability and silicon area is implemented for fair comparison. The BW-aware partition eDRAM, BL-segmented CIM with weight update scheduler, and MFCM together contribute to 8.07× performance and 3.69× system-efficiency improvement.

3) Macro and System FoMs: Prior works [24], [29], [30], [31] predominantly focus on peak compute throughput and energy efficiency, while overlooking on-chip bandwidth utilization. This limitation becomes more pronounced in dynamic bandwidth-demanding models such as the U-Net model architecture. To comprehensively evaluate the aforementioned limitations, we define new figures-of-merits (FoMs) for both system- and macro-level evaluation, as shown in Fig. 20. The system FoM is defined as the product of system energy efficiency and effective on-chip bandwidth, while the macro FoM is defined as the product of macro energy efficiency and macro area efficiency. Compared with the prior SOTA accelerator [30] in system FoM, *MTDiff* achieves a 1.44× improvement, primarily attributed to the proposed bandwidth-aware partitioned eDRAM architecture. In terms of macro FoM, *MTDiff* attains a 1.28× gain over the SOTA work [24], benefiting from the optimized CIM macro design.

C. Comparison With SOTA Works

Table II summarizes a detailed comparison with prior SOTA Diffusion accelerators and other digital CIM processors. Prior works can be categorized based on their target applications and primary architectural features: diffusion-based image generation accelerators, such as heterogeneous dense–sparse CIM-core architectures [24] and accelerators built upon energy-efficient PE arrays [29]; transformer and CNN accelerators, exemplified by an attention-token-bit hybrid sparse digital CIM-based accelerator [30] and intensive-CIM sparse-digital architecture [31]. In this landscape, *MTDiff* is distinguished by its tightly integrated digital CIM and eDRAM banks with configurable bandwidth partitioning, making it highly suited for workloads with dynamic compute and bandwidth demands, particularly in U-Net–based Diffusion models.

In contrast to conventional image generation accelerators [24], [29], which primarily optimize compute energy efficiency, *MTDiff* is specifically engineered for diffusion acceleration with bandwidth-aware allocation to optimize system energy efficiency. In addition to image generation, *MTDiff* leverages the scalability of diffusion models to support 3-D reconstruction and video generation tasks. Guo et al. [24] exploit activation data distribution by partitioning activations into a low-precision dense matrix and a high-precision sparse matrix, and employ two heterogeneous cores to support the corresponding computations. Nevertheless, although this approach improves performance by reducing the average bitwidth and computation, its efficiency gains are highly dependent on the data distribution. Consequently, the practical bandwidth of the heterogeneous cores may not fully satisfy computational demands, thereby limiting the achievable system energy efficiency. Qin et al. [29] propose an algorithm–hardware co-design that eliminates redundant computations in convolutional attention and adopts a bipolar floating-point Multiplication. However, the architecture only supports a conventional single FP16 precision, which imposes higher bandwidth requirements and increases pressure on the memory subsystem.

In contrast, *MTDiff* achieves 1.52× and 1.24× improvements in effective system efficiency compared to [24] and [29], respectively, under a unified image generation workload. To

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:20:29 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

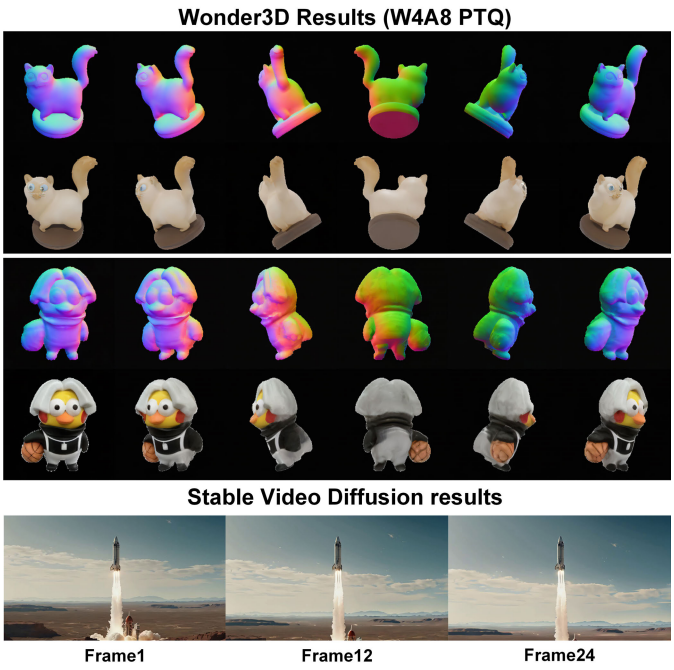


Fig. 21. Final generation results of *MTDiff*.

fairly reflect the joint benefits in bandwidth utilization, area efficiency, and energy efficiency, we further adopt the proposed system FoM and macro FoM for evaluation. *MTDiff* achieves 13.67× and 3.82× improvements in system FoM over [24] and [29], respectively, as well as a 1.28× improvement in macro FoM compared to [24]. Guo et al. [24] and Qin et al. [29] optimize only a single view at the pixel level. *MTDiff* performs hierarchical optimization for multi-view processing at both the frame level and the pixel level, enabling more efficient support for 3-D reconstruction and video generation tasks. As a result, *MTDiff* achieves 54.91 TFLOPS/W for 3-D reconstruction and 49.74 TFLOPS/W for video generation.

We also compare *MTDiff* with two digital CIM-based accelerators [30] and [31]. Although customized designs enable their digital CIM macros to provide high on-chip bandwidth, they rely on static bandwidth allocation schemes, which lead to either underutilization or insufficiency of on-chip bandwidth under dynamic workloads. *MTDiff* integrates high-density eDRAM to expand on-chip memory and reduce external memory accesses. For macro energy efficiency, the evaluation results demonstrate that *MTDiff* achieves 1.59× and 1.46× improvements over [30] and [31], respectively.

Fig. 21 presents the final generation results of *MTDiff*. We adopt Wonder3D and stable video diffusion as evaluation benchmarks. For the Wonder3D results, both the normal maps and RGB images are illustrated. These outputs are subsequently fused through a geometry-aware reconstruction pipeline to synthesize the final 3-D model. For the stable video diffusion results, a single context frame is provided as input to generate the corresponding video sequence, from which three representative frames are sampled to demonstrate generation quality. All models are deployed with the proposed mixed-precision quantization strategy, and their respective training datasets are adopted as calibration sets for post-training quantization.

VIII. Conclusion

This article introduces *MTDiff*, a diffusion accelerator designed for efficient multi-task content generation. Our approach achieves maximum performance and efficiency through cross-hierarchical innovations. At the architectural level, dynamic bandwidth-aware memory partitioning with eDRAM storage enhances utilization and reduces external memory access. At the circuit level, a BL-segmented CIM with a reuse-aware weight update scheduler establishes a system-friendly acceleration subsystem. Additionally, a hierarchical MFCM unit further optimizes performance and efficiency at the algorithmic level. Fabricated in 22-nm technology, the *MTDiff* processor achieves a system efficiency of 49.74–60.81 TFLOPS/W across diverse generation tasks. As the first CIM-based diffusion accelerator to support multi-content generation, it provides a 1.4× performance improvement and a 1.52× enhancement in system energy efficiency compared to the prior diffusion accelerators.

References

[1] Y. Cao et al., "A survey of AI-generated content (AIGC)," *ACM Comput. Surv.*, vol. 57, no. 5, pp. 1–38, May 2025.
[2] A. Ramesh et al., "Zero-shot text-to-image generation," in *Proc. Int. Conf. Mach. Learn. (ICML)*, Jul. 2021, pp. 8821–8831.
[3] Y. Liu et al., "Sora: A review on background, technology, limitations, and opportunities of large vision models," 2024, *arXiv:2402.17177*.
[4] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *Proc. NIPS*, vol. 33. Vancouver, BC, Canada: Curran Associates, 2020, pp. 6840–6851.
[5] J. Song, C. Meng, and S. Ermon, "Denoising diffusion implicit models," 2020, *arXiv:2010.02502*.
[6] A. Ramesh, P. Dhariwal, A. Nichol, C. Chu, and M. Chen, "Hierarchical text-conditional image generation with CLIP latents," 2022, *arXiv:2204.06125*.
[7] J. Betker et al., "Improving image generation with better captions," *Comput. Sci.*, vol. 2, no. 3, p. 8, 2023. [Online]. Available: https://cdn.openai.com/papers/dall-e-3.pdf
[8] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2022, pp. 10684–10695.
[9] C. Wang, H.-Y. Peng, Y.-T. Liu, J. Gu, and S.-M. Hu, "Diffusion models for 3D generation: A survey," *Comput. Vis. Media*, vol. 11, no. 1, pp. 1–28, Feb. 2025.
[10] Y. Jiang, C. Yu, C. Cao, F. Wang, W. Hu, and J. Gao, "Animate3D: Animating any 3D model with multi-view video diffusion," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 125879–125906.
[11] J. Ho et al., "Imagen video: High definition video generation with diffusion models," 2022, *arXiv:2210.02303*.
[12] Z. Yang et al., "CogVideoX: Text-to-video diffusion models with an expert transformer," 2024, *arXiv:2408.06072*.
[13] D. P Kingma and M. Welling, "Auto-encoding variational Bayes," 2013, *arXiv:1312.6114*.
[14] I. J. Goodfellow et al., "Generative adversarial nets," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 27, 2014, pp. 1–11.
[15] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in *Proc. 18th Int. Conf. Med. Image Comput. Comput.-Assist. Intervent.*, vol. 9351. Cham, Switzerland: Springer, 2015, pp. 234–241.
[16] W. Peebles and S. Xie, "Scalable diffusion models with transformers," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2023, pp. 4195–4205.
[17] W. Tang and H. Zhao, "Score-based diffusion models via stochastic differential equations," *Statist. Surv.*, vol. 19, pp. 28–64, Jan. 2025.
[18] Y. Jing et al., "37.6 A 22 nm 60.81TFLOPS/W diffusion accelerator with bandwidth-aware memory partition and BL-segmented compute-in-memory for efficient multi-task content generation," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2025, pp. 1–3.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:20:29 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

[19] P. Guo, T. Hao, W. Fu, Y. Gao, and Y. Li, "Controllable 3D dance generation using diffusion-based transformer U-Net," in *Proc. AAAI Conf. Artif. Intell.*, vol. 39, no. 3, 2025, pp. 3284–3292.
[20] X. Wang, Z. He, and Y. Zhou, "Dynamic importance in diffusion U-Net for enhanced image synthesis," in *Proc. IEEE Int. Conf. Multimedia Expo. (ICME)*, Jun. 2025, pp. 1–6.
[21] M. Waseem, M. U. G. Khan, and S. K. Khurshid, "LCGD: Enhancing text-to-video generation via contextual LLM guidance and U-Net denoising," *IEEE Access*, vol. 13, pp. 47068–47085, 2025.
[22] X. Long et al., "Wonder3D: Single image to 3D using cross-domain diffusion," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2024, pp. 9970–9980.
[23] Y. Wang et al., "34.1 A 28 nm 83.23TFLOPS/W POSIT-based compute-in-memory macro for high-accuracy AI applications," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2024, pp. 566–568.
[24] R. Guo et al., "20.2 A 28 nm 74.34TFLOPS/W BF16 heterogenous CIM-based accelerator exploiting denoising-similarity for diffusion models," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2024, pp. 362–364.
[25] C. Schuhmann et al., "LAION-5B: An open large-scale dataset for training next generation image-text models," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 25278–25294.
[26] M. Deitke et al., "Objaverse: A universe of annotated 3D objects," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2023, pp. 13142–13153.
[27] A. Blattmann et al., "Stable video diffusion: Scaling latent video diffusion models to large datasets," 2023, *arXiv:2311.15127*.
[28] T.-S. Chen et al., "Panda-70M: Captioning 70M videos with multiple cross-modality teachers," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2024, pp. 13320–13331.
[29] Y. Qin et al., "A 52.01 TFLOPS/W diffusion model processor with inter-time-step convolution-attention-redundancy elimination and bipolar floating-point multiplication," in *Proc. IEEE Symp. VLSI Technol. Circuits (VLSI Technol. Circuits)*, Jun. 2024, pp. 1–2.
[30] F. Tu et al., "16.1 MuITCIM: A 28nm 2.24μJ/token attention-token-bit hybrid sparse digital CIM-based accelerator for multimodal transformers," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2023, pp. 248–250.
[31] J. Yue et al., "A 28 nm 16.9-300TOPS/W computing-in-memory processor supporting floating-point NN inference/training with intensive-CIM sparse-digital architecture," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2023, pp. 1–3.



Jiaqi Zhou received the B.S. degree from the School of Integrated Circuits, Shandong University, Jinan, Shandong, China, in 2024. He is currently pursuing the Ph.D. degree with the School of Integrated Circuits, Peking University, Beijing, China.

His research interests include CIM-based heterogeneous AI accelerators for the field of large generative models and acceleration algorithms for AI.



Yiqi Jing (Graduate Student Member, IEEE) received the B.E. degree from the School of Microelectronics, Tianjin University, Tianjin, China, in 2021. He is currently pursuing the Ph.D. degree with the School of Integrated Circuits, Peking University, Beijing, China.

His research interests include AI accelerators for domain-specific applications and reconfigurable and flexible architectures for AI.



Yiyang Sun received the B.S. degree from the School of EECS, Peking University, Beijing, China, in 2023, where he is currently pursuing the Ph.D. degree with the School of Integrated Circuits.

His research interests include AI accelerator design for computer vision.



Siyuan He (Graduate Student Member, IEEE) received the B.S. degree from the Huazhong University of Science and Technology, Wuhan, China, in 2023. He is currently pursuing the Ph.D. degree with the School of Integrated Circuits, Peking University, Beijing, China.

His research interests include domain-specific accelerators and architecture for AI, and processing-in-memory technology.



Le Ye (Senior Member, IEEE) was born in Jiangsu, China, in 1983. He received the B.S. degree (Hons.) in physics from the Department of Physics, Nanjing University, Nanjing, China, in 2005, and the Ph.D. degree in microelectronics from the Institute of Microelectronics, Peking University, Beijing, China, in 2010.

From April 2009 to April 2010, he was a Visiting Scholar at the Institute of Circuits and Systems, Dresden University of Technology, Dresden, Germany. Since 2010, he has been a Faculty Member with Peking University, where he is currently an Associate Professor. He is currently the Dean of the IoT Research Center, Advanced Institute of Information Technology, Peking University, Hangzhou, China. He has authored or co-authored more than 40 articles. He is the holder of more than 20 granted patents. His research interests include analog and mixed-signal IC design, ultralow power IC design, computing-in-memory AI chip design, and implanted IC and micro-system for medical applications.

Dr. Ye was a recipient of the National Science Fund for Outstanding Young Scholars and Beijing New-Star Plan of Science and Technology in China. He was an Associate Editor of IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS—I: REGULAR PAPERS in 2020.



Tianyu Jia (Member, IEEE) received the B.S. and M.S. degrees from Beijing University of Posts and Telecommunications, Beijing, China, and the Ph.D. degree in computer engineering from Northwestern University, Evanston, IL, USA, in 2019.

He was a Post-Doctoral Fellow at Harvard University, Cambridge, MA, USA, and an Assistant Research Professor at Carnegie Mellon University, Pittsburgh, PA, USA. He is currently an Assistant Professor at the School of Integrated Circuits, Peking University, Beijing. His research interests include domain-specific AI architecture, compute/processing-in-memory, and heterogeneous SoC design.

Dr. Jia received the IEEE SSCS Predoctoral Achievement Award in 2020 and the Best Paper Award from 2025 ISLPED and 2026 ISEDA. He currently serve as an Associate Editor of IEEE Computer Architecture Letters and TPC member for ICCAD, DATE, MICRO and HPCA.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:20:29 UTC from IEEE Xplore. Restrictions apply.